

Curs 4 — Structuri de Date (Gabriel Istrate)

Sinteză: Tabele de dispersie (Hash Tables)

🎯 = punct de examen, ⚠️ = capcană, 🌱 = exercițiu „de discutat” rezolvat.

Notă: PDF-ul acoperă partea de hashing din Cursul 4 (slide-urile 23–50). Slide-ul cu santinela ($(L.nil)$) atașat separat e din Cursul 2 — vezi §8.2 din sinteza Cursului 2 (lista dublu înlănțuită cu nod fictiv).

0. De ce e cursul cel mai important pentru grilă

Hashing-ul apare pe **fiecare** grilă din harta ta de examene. Tipurile de întrebări pe care acest curs ți le pregătește direct:

Tip de întrebare de examen	Unde apare	Ce-ți trebuie din curs
Maparea cu chaining ($(h(x)=x \bmod m)$, ce coliziuni/lanțuri)	model Q5, grila2025 Q5, grila2023 Q6	metoda diviziunii + chaining
Double hashing (insert + ce celule rămân goale)	grila2025 Q18, toamnă Q2	$(h(k,i)=(h_1+i \cdot h_2) \bmod m)$, permutație
Load factor (definiție + comportament la dublare/triplare)	grila2025 Q12, model Q11, grila2023 Q12	$\alpha = n/m$
Afirmații despre load factor (între 0 și 1?)	grila2025 Q12	chaining: α poate >1 ; open addr.: $\alpha \leq 1$
Funcții alternative de double hashing potrivite?	model Q17	h_2 coprîm cu m , permutație
Complexitatea căutării	conceptual	chaining $\Theta(1+\alpha)$

Reține înainte de orice altceva: definiția lui α , distincția chaining vs open addressing, și condiția de coprimăritate a lui h_2 . Astea trei acoperă majoritatea întrebărilor.

1. Ideea de bază

Problema: universul cheilor U e uriaș, dar tabelul cu adresare directă (din Cursul 2) ar cere spațiu $\Theta(|U|)$ — risipă. Vrem accesul $O(1)$ **fără** memorie cât $|U|$.

Soluția — hash table:

- folosim un tabel T cu $|T| \ll |U|$;
- mapăm fiecare cheie $k \in U$ la o poziție în T cu o **funcție hash** $h : U \rightarrow \{1, \dots, |T|\}$;
- $h(k)$ trebuie ușor de calculat — $O(1)$ dat k .

Operațiile naive:

```
Hash-Insert(T, k): T[h(k)] = true
Hash-Delete(T, k): T[h(k)] = false
Hash-Search(T, k): return T[h(k)]
```

⚠ **Sunt mereu corecte? NU!** Ce se întâmplă dacă două chei distincte ($k_1 \neq k_2$) produc o **coliziune** ($h(k_1) = h(k_2)$)? Atunci se suprascriu / se confundă. Tot restul cursului = cum tratăm coliziunile.

2. Intuiția „on average” (bile în coșuri)

Hash table-urile merg bine **în medie**:

- **Analogie:** arunci T bile la întâmplare în N coșuri.
 - Dacă $T \ll N$ (mai exact $T = o(\sqrt{N})$), atunci cu probabilitate mare **nicio** pereche de bile nu cade în același coș (e granița paradoxului zilei de naștere).
 - În medie: T/N bile în fiecare coș.
 - Vrem ca funcția hash să fie „random-like”: elementele lui U „aruncate uniform” de h peste pozițiile lui T . Asta e ipoteza de **uniform hashing**.
-

3. Două strategii de rezolvare a coliziunilor

1. **Chaining (înlănțuire):** fiecare poziție din tabel ține o **listă înlănțuită** cu toate cheile care se mapează acolo. (α poate fi orice.)
 2. **Open addressing (adresare deschisă):** stochezi **toate** elementele direct în tabel; la coliziune cauți altă celulă liberă. (Forțează $\alpha \leq 1$.)
-

4. Chaining (înlănțuire)

Idee: stochezi toate obiectele care se mapează la același bucket într-o **listă înlănțuită**.

„Speri” că funcția hash e suficient de uniformă, listele nu sunt prea lungi, operațiile rămân eficiente.

Operații (deleagă către lista din bucket):

```
Chained-Hash-Insert(T, k): return List-Insert(T[h(k)], k)
```

```
Chained-Hash-Search(T, k): return List-Search(T[h(k)], k)
```

4.1 Load factor (factorul de încărcare) — definiția cheie

$$\alpha = \frac{n}{|T|}$$

unde **n** = numărul de elemente stocate, **|T|** = numărul de poziții (sloturi) din tabel.

🌀 Asta e definiția testată la fiecare grilă. Reține și consecințele:

- α se **dublează** dacă **n** se dublează (**|T|** fix).
- α se **înjumătățește** dacă **|T|** se dublează (**n** fix).
- α devine $(2/3) \cdot \alpha$ dacă **|T|** se triplează și **n** se dublează ($2n/3m = (2/3)(n/m)$).
- ⚠️ **La chaining, α poate fi > 1** (o listă poate ține oricâte elemente). Asta e crucial pentru întrebarea „ α e între 0 și 1?”.

4.2 Analiza (sub ipoteza uniform hashing)

- $\Pr[h(k) = i] = 1/|T|$ pentru orice poziție **i**.
- Pentru **n** chei distincte, lungimea **medie** a listei din poziția **i**:

$$E[n_i] = \frac{n}{|T|} = \alpha$$

- Presupunem că $h(k)$ se calculează în $O(1)$.
- 🌀 **Complexitatea lui Chained-Hash-Search:**

$$\Theta(1 + \alpha)$$

(„1” = calculul lui $h(k)$ + accesul la slot; „ α ” = parcurgerea listei de lungime medie α).

5. Open Addressing (adresare deschisă)

Idee: în loc de liste înlănțuite, stochezi **toate** elementele chiar în tabel.

- ⚠️ Asta implică $\alpha \leq 1$ (nu poți pune mai multe elemente decât sloturi). 🌀 Asta e răspunsul la „ α e între 0 și 1 dacă folosim open addressing” → ADEVĂRAT.
- La o coliziune, cauți pur și simplu **altă celulă liberă** în T, urmând o **secvență de sondare** (probe sequence).

5.1 Secvența de sondare și algoritmii

Funcția hash devine $h(k, i)$ — al i-lea slot încercat pentru cheia k.

Hash-Insert(T, k):

```

1  for i = 1 to T.length
2      j = h(k, i)
3      if T[j] == nil
4          T[j] = k
5          return j
6  error "overflow"
```

Hash-Search(T, k):

```

1  i = 0
2  repeat
3      j = h(k, i)
4      if T[j] == k: return j
5      i = i + 1
6  until T[j] == nil or i == m
7  return nil
```

Înțelegere: încerci sloturile $h(k, 0), h(k, 1), \dots$ în ordine. La inserare te oprești la primul slot liber. La căutare te oprești fie când găsești k, fie când dai de un slot **gol** (NIL → k nu există), fie când ai epuizat toate sloturile.

⚠️ **Condiție esențială:** $h(k, \cdot)$ trebuie să fie o **permutație** a tabelului — adică $h(k, 1), h(k, 2), \dots, h(k, |T|)$ trebuie să acopere **toate** sloturile. Altfel un element ar putea să nu fie inserabil deși există loc liber. 🌀 Asta e fix principiul din spatele întrebării model Q17 (funcțiile alternative de double hashing eșuează pentru că nu generează o permutație completă).

5.2 ⚠️ Ștergerea e dificilă în open addressing

- A marca slotul cu **NIL nu funcționează**: ai putea face imposibilă regăsirea unei chei a cărei inserare a trecut prin slotul i și l-a găsit ocupat. (Căutarea s-ar opri la NIL crezând că lista s-a terminat, deși cheia e mai departe în secvență.)

- **Soluție:** un marcaj special **DELETED** în loc de NIL. Căutarea îl tratează ca „ocupat, continuă sondarea”, inserarea ca „liber, pot pune aici”.
- **Problemă:** timpul de căutare nu mai depinde doar de load factor (sloturile DELETED se acumulează).

5.3 🌸 Întrebare „de discutat” din curs (fără răspuns) — REZOLVATĂ

„O metodă secvențială de sondare poate să nu fie optimă — îți poți imagina de ce?”

Răspuns: din cauza fenomenului de **clustering primar (primary clustering)**. La sondarea secvențială/liniară, când mai multe chei aterizează lângă aceeași zonă, se formează **șiruri lungi de sloturi ocupate consecutiv**. Orice cheie nouă care se mapează oriunde în interiorul (sau înaintea) unui astfel de cluster e „absorbită” în el și trebuie să sondeze tot șirul până la primul slot liber. Clusterelor lungi tind să crească și mai repede (un cluster mai lung „prinde” mai multe chei), deci **timpul mediu de căutare crește**. Vrem secvențe de sondare care „împrăștie” mai bine — de aici quadratic probing și, mai ales, double hashing.

5.4 Tehnici de sondare (probing)

(a) **Linear probing (sondare liniară):**

$$h(k, i) = (h'(k) + i) \bmod m$$

unde h' e o funcție hash auxiliară. Ușor de implementat, dar suferă de **primary clustering** (vezi mai sus): se formează șiruri lungi de sloturi ocupate.

(b) **Quadratic probing (sondare pătratică):**

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

Merge mult mai bine decât cea liniară, dar valorile lui (c_1, c_2, m) sunt **constrânse** ca să acopere tot tabelul. Suferă de **secondary clustering**: dacă două chei au aceeași poziție inițială de sondare, au **aceeași** secvență de sondare în continuare.

(c) **Double hashing (dispersie dublă) — printre cele mai bune metode:**

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

- 🌀 $h_2(k)$ trebuie să fie **relativ prim (coprim) cu m** — altfel secvența nu acoperă tot tabelul (nu e permutație). Soluții practice:
 - m putere a lui 2 și $h_2(k)$ impar; sau
 - m prim și $h_2(k) < m$.
- Evită atât clustering-ul primar cât și pe cel secundar (fiecare cheie are pasul ei $h_2(k)$).

5.5 Complexitatea căutării în open addressing

🌀 Dat un tabel cu open addressing și load factor $\alpha = n/m < 1$, numărul **mediu** de sondări într-o căutare **nereușită** e cel mult:

$$\frac{1}{1 - \alpha}$$

(sub ipoteza uniform hashing). Ex: $\alpha = 0.5 \rightarrow \leq 2$ sondări; $\alpha = 0.9 \rightarrow \leq 10$ sondări. Cu cât tabelul e mai plin, cu atât mai scump.

6. Funcții hash bune

⚠ Domeniul e uriaș; multe funcții sunt bune doar pentru scopuri specifice — nu le folosi orbește. Cursul urmează Cormen (idei generale).

Cerință: o funcție hash bună satisface (aproximativ) ipoteza de **uniform hashing**. Din păcate, de obicei nu cunoaștem distribuția cheilor și cheile pot să nu fie independente.

- **Cazul ideal:** dacă cheile sunt reale aleatoare uniform în $[0,1)$, atunci $h(k) = \lfloor k \cdot m \rfloor$ satisface uniform hashing-ul simplu.
- **De obicei** presupunem că universul cheilor = numerele naturale. Ex: un șir de caractere = un întreg în baza 128. Identificatorul „pt” (ASCII p=112, t=116) $\rightarrow \lfloor 112 \cdot 128 + 116 \rfloor = 14452$.

6.1 Metoda diviziunii (cea de pe grile)

$$h(k) = k \bmod m$$

- ⚠ **Evită anumite valori ale lui m**, în special **puterile lui 2**: dacă $m = 2^p$, atunci $h(k)$ = cei mai puțin semnificativi p biți ai lui k — prost dacă acei biți nu sunt uniformi.
- **Un număr prim, nu prea aproape de o putere a lui 2, e adesea o alegere bună.**

🌀 Toate întrebările de chaining de pe grile folosesc exact $h(x) = x \bmod m$ ($\bmod 10$, $\bmod 11$, $\bmod 100$). Vezi §11.1 pentru trace-uri.

6.2 Folding (împăturire)

Spargi intrarea în bucăți, le combini. **Exemplu (SSN american):** $123456789 \rightarrow$ împarți în $123, 456, 789 \rightarrow$ aduni $\rightarrow 1368 \rightarrow \bmod \text{table-size } (1000) \rightarrow 368$.

6.3 Metoda înmulțirii (multiplication method)

Procedură în două etape:

$$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$$

1. înmulțești cheia cu o constantă A , $(0 < A < 1)$, și extragi **partea fracționară** $(\{kA \bmod 1\})$;
 2. înmulțești cu m și iei partea întreagă (floor).
- Valoarea lui m nu e critică; tipic $m = 2^p$.
 - Implementare ușoară: $A = s / 2^w$ (w = dimensiunea cuvântului mașinii).
 - Knuth sugerează $A \approx (\sqrt{5} - 1)/2$ (raportul de aur) — merge bine.
-

7. Universal hashing (dispersie universală)

Motivație — atacul în cazul defavorabil:

- Orice funcție hash **fixă** e vulnerabilă: dacă un „adversar” alege n chei care se mapează **toate** la același slot, căutarea medie devine $\Theta(n)$ (tabelul degenerază în listă).
- Practic: Crosby & Wallach (USENIX'03) au arătat atacuri DoS care încetinesc până la blocare sisteme reale (hash table-urile din Perl, proxy-ul web Squid, sistemul de detecție a intruziunilor Bro) — **cauza: mecanismul de hashing era cunoscut** (implementare publică).
- **Soluție:** alegeți funcția hash **aleator**, independent de cheile care vor fi stocate.

Definiția familiei universale:

- $\mathcal{H}$ = colecție finită de funcții hash care mapează U în $\{0, 1, \dots, m-1\}$.
- $\mathcal{H}$ e **universală** dacă pentru orice chei $k \neq l \in U$, numărul de funcții $h \in \mathcal{H}$ cu $h(k) = h(l)$ e cel mult $|\mathcal{H}|/m$. (Echivalent: pentru un h ales aleator, $P(h(k)=h(l)) \leq 1/m$.)
- Cu chaining, folosind un h dintr-o familie universală pentru n chei într-un tabel de dimensiune m :
 - dacă cheia **NU** e în tabel, lungimea medie a listei la care se mapează e $\leq \alpha$;
 - dacă cheia **ESTE** în tabel, lungimea medie e $\leq 1 + \alpha$.
 - ⚠ (Slide-ul are un typo — repetă „If key is not in the table” de două ori; corect e: $not\ in \rightarrow \alpha, in \rightarrow 1+\alpha$.)

7.1 O familie universală concretă (Carter-Wegman)

- $a \equiv b \pmod{p}$ înseamnă $p \mid (a - b)$.
- $\mathbb{Z}_p$ = întregii modulo p , cu p **prim**, ales astfel încât toate cheile să fie în $\{0 \dots p-1\}$.
- m = numărul de sloturi; $a \in \mathbb{Z}_p^*$ (nenul), $b \in \mathbb{Z}_p$.

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$$

$$H_{p,m} = \{h_{a,b} : a \in \mathbb{Z}_p^*, b \in \mathbb{Z}_p\}$$

Această familie e universală. (Aceleași funcții apar și la generatoare pseudo-aleatoare.)

8. Perfect hashing (dispersie perfectă)

- Hashing-ul poate da performanță **garantată în cazul defavorabil** când mulțimea de chei e **statică** (nu se mai schimbă după ce a fost stocată). Ex: fișierele de pe un DVD-R finalizat.
 - **Perfect hashing**: numărul de accese în cazul defavorabil pentru o căutare e $O(1)$.
 - **Idee**: dispersie pe **două niveluri**, cu hashing universal la fiecare nivel.
 - **Nivelul 1**: cele n chei se mapează în m sloturi cu o funcție universală.
 - În loc de chaining: la fiecare slot j folosești un **tabel secundar mic** S_j cu propria funcție h_j .
 - Alegând h_j cu grijă, **garantezi zero coliziuni** la nivelul 2. (Standard: dimensiunea tabelului secundar pătratică în numărul de chei care cad acolo elimină coliziunile; spațiul total rămâne $O(n)$ în medie.)
-

9. Mai departe (cultură generală — rar de examen, dar bine de știut)

- **Funcții hash criptografice** (proprietăți de securitate): md5, sha-1, sha-2, sha-3 (standarde guvernamentale SUA). Există atacuri pe sha-1 (CWI Amsterdam, 2017).
- **Bloom filters**: structură **probabilistă** pentru testarea apartenenței.
 - răspuns **NU** → întotdeauna corect (fără false negative);
 - răspuns **DA** → posibil **fals pozitiv**.
- **MapReduce** (calcul pe grid): programare cu funcții hash; date = perechi cheie-valoare; *Map* aplicat în paralel produce perechi cheiate cu k_2 ; sistemul grupează după k_2 ; *Reduce* aplicat pe fiecare grup.
- **Locality-Sensitive Hashing (LSH)**: reduce dimensionalitatea datelor; elementele **similare** ajung în aceleași bucket-uri cu probabilitate mare.

- **În limbaje:** Python — dicționarele; C++ STL — `std::hash`, `std::unordered_map`, `std::unordered_set`.

10. Cheat-sheet — fapte de examen

Concept	De reținut
Load factor	$\alpha = n/m$
α când n se dublează	se dublează
α când	T
α când	T
Chaining — domeniul lui α	$[0, \infty)$, poate fi > 1
Open addressing — domeniul lui α	$\alpha \leq 1$
„ α între 0 și 1?”	adevărat doar pentru open addressing
Căutare chaining	$\Theta(1 + \alpha)$
Căutare nereușită open addr.	$\leq 1/(1 - \alpha)$ sondări
Metoda diviziunii	$h(k) = k \bmod m$; evită puterile lui 2; prim = bun
Metoda înmulțirii	$h(k) = \lfloor m(kA \bmod 1) \rfloor$, $A \approx (\sqrt{5}-1)/2$
Double hashing	$h(k, i) = (h_1 + i \cdot h_2) \bmod m$
Condiția double hashing	$h_2(k)$ coprim cu m (sau $m=2^p$ & h_2 impar; sau m prim & $h_2 < m$)
$h(k, \cdot)$ în open addr.	trebuie permutație (acoperă tot tabelul)
Linear probing	$(h'(k) + i) \bmod m \rightarrow$ clustering primar
Quadratic probing	$(h'(k) + c_1 i + c_2 i^2) \bmod m \rightarrow$ clustering secundar
Ștergere open addr.	nu cu NIL $\rightarrow$ folosește DELETED
Universal hashing	familie cu $ \{h: h(k)=h(l)\} \leq$

Concept	De reținut
Carter-Wegman	$h_{\{a,b\}}(k) = ((ak+b) \bmod p) \bmod m$, p prim
Perfect hashing	chei statice , două niveluri, worst-case O(1)
Bloom filter	NU = sigur; DA = posibil fals pozitiv

11. Lucrate pas cu pas (exemple de tip examen)

11.1 Chaining cu metoda diviziunii

Set: (4322, 1334, 1471, 9679, 1989, 6171, 6173, 4199).

Cu $h(x) = x \bmod 10$ (ultima cifră) — *model Q5*:

Slot	Lanț
1	1471 → 6171
2	4322
3	6173
4	1334
9	9679 → 1989 → 4199

Cu $h(x) = x \bmod 100$ (ultimele două cifre) — *grila2023 Q6*:

- 1471→71, 6171→71 → **coliziune** (afirmația (i) adevărată);
- 9679→79, 1989→89, 4199→99 → **toate diferite** (afirmația (ii) falsă);
- 8 elemente / 100 sloturi → $\alpha = 8\%$ (deci „7%” e fals).
- ⚠ Morala: aceeași listă de numere, dar $\bmod 10$ vs $\bmod 100$ schimbă complet coliziunile (compari câte cifre finale).

11.2 Double hashing

Insert 27, 43, 692, 72 în tabel de dimensiune $m = 7$, cu $h_1(k) = k \bmod 7$, $h_2(k) = 1 + (k \bmod 5)$ — *grila2025 Q18 / toamnă Q2*:

- **27:** $h_1 = 6 \rightarrow$ slot **6**.
- **43:** $h_1 = 1 \rightarrow$ slot **1**.

- **692:** $h_1 = 6$ (ocupat). $h_2 = 1 + (692 \bmod 5) = 1+2 = 3$. $\boxed{(6+3) \bmod 7 = 2} \rightarrow \text{slot } 2$.
- **72:** $h_1 = 2$ (ocupat). $h_2 = 1 + (72 \bmod 5) = 1+2 = 3$. $\boxed{(2+3) \bmod 7 = 5} \rightarrow \text{slot } 5$.
- Ocupate: 1, 2, 5, 6 $\rightarrow$ **goale: 0, 3, 4**.
- (Verifică coprimăitatea: $h_2 \in \{1,2,3,4,5\}$ și $m=7$ prim $\rightarrow$ orice $h_2 < 7$ e coprim cu 7, deci secvența e mereu o permutație validă.)

11.3 De ce funcțiile alternative de double hashing eșuează

model Q17: date $H_1(x, i) = (h_2(x) + i \cdot h_1(x)) \bmod m$ și $H_2(x, i) = (h_2(x) - 1 + i \cdot (h_1(x)+1)) \bmod m$, niciuna nu e potrivită:

- **H_1 :** ca să acopere tot tabelul (permutație), „pasul” $\boxed{h_1(x)}$ ar trebui să fie nenul și coprim cu m — nu e garantat $\rightarrow$ pot rămâne sloturi nevizitate.
- **H_2 :** prin construcția pasului $\boxed{h_1(x)+1}$, sare peste anumite sloturi $\rightarrow$ nu e permutație.
- Principiul din curs: **pasul (al doilea termen) trebuie coprim cu m** , altfel secvența de sondare nu e o permutație a tabelului (§5.1, §5.4c).

11.4 Load factor — comportament

- Sloturi se dublează (n fix): $\alpha \rightarrow \alpha/2$ (*model Q11*).
- Sloturi $\times 3$, elemente $\times 2$: $\alpha \rightarrow (2/3)\alpha$ (*grila2023 Q12*).
- Elemente se dublează (sloturi fixe): $\alpha \rightarrow 2\alpha$ (*grila2025 Q12 (d)*).
- „ α e între 0 și 1?”: doar cu **open addressing** (*grila2025 Q12 (b)*); cu chaining poate depăși 1.

12. Recapitulare a întrebărilor „de discutat” rezolvate

1. 🌱 „Sunt algoritmi naivi mereu corecți?” $\rightarrow$ NU, din cauza coliziunilor (§1).
2. 🌱 „De ce nu e optimală sondarea secvențială?” $\rightarrow$ din cauza **clustering-ului primar**: se formează șiruri lungi de sloturi ocupate care cresc timpul mediu de căutare (§5.3).
3. ⚠️ **Typo corectat** la universal hashing: cheie absentă $\rightarrow$ listă medie $\leq \alpha$; cheie prezentă $\rightarrow \leq 1+\alpha$ (§7).

Bottom line: din acest curs, pentru grilă trebuie să ieși cu (1) definiția $\alpha = n/|T|$ și toate regulile ei la dublare/triplare; (2) chaining: α poate > 1 , căutare $\Theta(1+\alpha)$; open addressing: $\alpha \leq 1$, căutare nereușită $\leq 1/(1-\alpha)$; (3) double hashing $\boxed{(h_1+i \cdot h_2) \bmod m}$ cu h_2 COPRIM cu m și secvența = permutație — asta e și cheia întrebărilor cu funcții alternative; (4) metoda diviziunii $\boxed{k \bmod m}$ pentru toate trace-urile de chaining (atenție mod 10 vs mod 100); (5)

ștergerea în open addressing cere *DELETED*, nu *NIL*. Restul (universal/perfect hashing, Bloom, MapReduce) e teorie de fundal — bine de înțeles, rar întrebat direct.